DSTL- Unit 5

Definition of a Graph

A graph is a collection of vertices (nodes) and edges (links) that connect pairs of vertices.

Formally,
A graph G is defined as

$$G = (V, E)$$

where

- V = set of vertices
- E = set of edges, where each edge connects two vertices (u, v)

  Note- Set V can not be empty but E can be empty set.

Types of Graphs

1. **Undirected Graph**

   Edges have no direction.

   Example: edge (u, v) is the same as (v, u).

2. **Directed Graph (Digraph)**

   Each edge has a direction from one vertex to another.

   Example: edge (u, v) ≠ (v, u).

3. **Weighted Graph**

   Each edge has an associated **weight or cost**.

4. **Unweighted Graph**

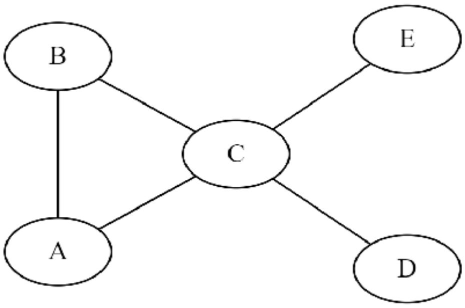
   All edges are considered to have equal weight.

5. **Connected Graph**

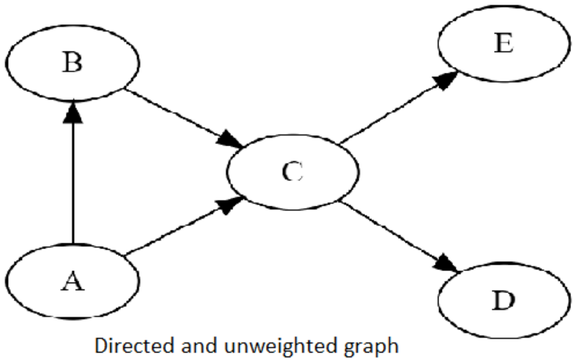
   There is a path between any two pair of vertices (u, v)

6. **Disconnected Graph**

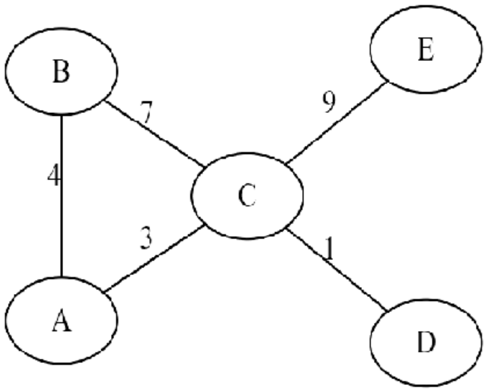
   For some pair of vertices (u, v), there is no path between them.



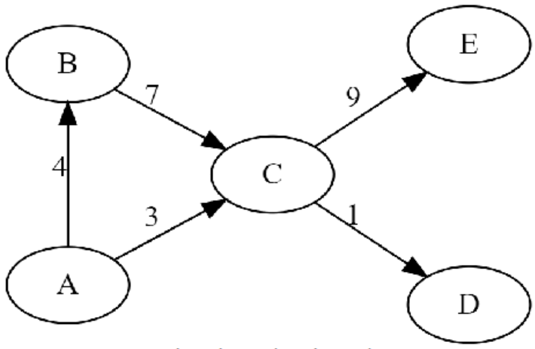
Undirected and unweighted graph



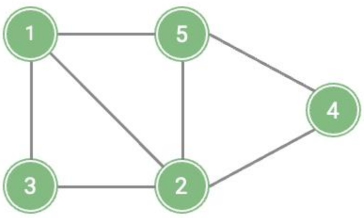
Directed and unweighted graph



Undirected and weighted graph

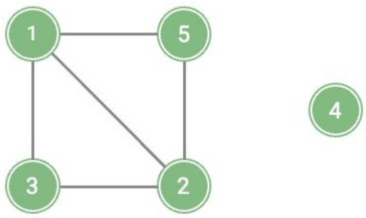


Directed and weighted graph



Connected graph

(All vertices are reachable)



Disconnected graph

(Some vertices are unreachable)

Basic Graph Terminology

Term	Meaning
Vertex (Node)	A point in a graph.
Edge	A line connecting two vertices.
Adjacent vertices	Two vertices connected by an edge.
Incident edge	An edge that touches a vertex.
Degree of vertex (deg(v))	Number of edges incident on a vertex.
In-degree / Out-degree	(For directed graphs) – number of incoming / outgoing edges of a vertex.
Parallel edges	Two or more edges connecting the same pair of vertices.
Loop	An edge that connects a vertex to itself.
Simple Graph	A graph with no loops or parallel edges.
Multigraph	A graph that may have parallel edges.
Null Graph	A graph with vertices but no edges.

Path, Walk, and Circuit in Graph

Term	Definition
Walk	A sequence of vertices and edges where each edge's endpoints are consecutive vertices.
Trail	A walk with **no repeated edges**.
Path	A walk with **no repeated vertices**.
Cycle / Circuit	A path that starts and ends at the same vertex.

Some Special Types of Graphs

Type	Description
Complete Graph (K_n)	Every vertex is connected to every other vertex by a direct edge. Therefore, it Has exactly $\frac{n(n-1)}{2}$edges.
Regular Graph	All vertices have the same degree.
Bipartite Graph	Vertices can be divided into two disjoint sets such that edges connect only between sets, not within.
Planar Graph	Can be drawn on a plane without edges crossing.
Null Graph	Graph with no edges.
Tree	A connected acyclic graph.

Note- In Complete Bi-partite Graph denoted as $K_{m,n}$, total vertices are m+n and total edges are exactly m*n.

Important Theorems Related with Graph

- **Degree Sum Theorem:**

Sum of degree of all the vertices is twice the number of edges.

$$\sum_{v \in V} deg(v) = 2 \mid E \mid$$

(Sum of degrees of all vertices = twice the number of edges)

- **In directed graph:**

$$\sum in\_degree(v) = \sum out\_degree(v) = \mid E \mid$$

Here $\mid E \mid$ is total number of edges.

- **Euler's Theorem (or Euler's Formula) for Planar Graphs**

For any connected planar graph,

$$\boxed{\boldsymbol{V - E + R = 2}}$$

where:

V = Number of vertices

E = Number of edges

R (or F) = Number of regions (including the *outer* region)

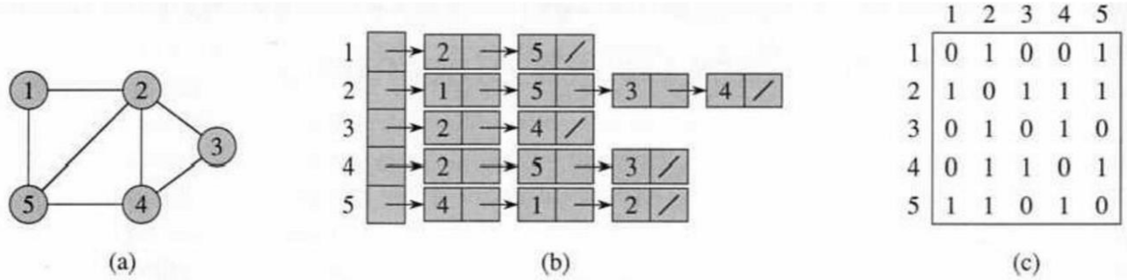
Representation of Graphs

A **graph** $G = (V, E)$can be represented in computer memory in various ways depending on how efficiently we want to store and process it.
The most common representations are:

1. **Adjacency Matrix :** size of matrix is v*v , where v is total vertices

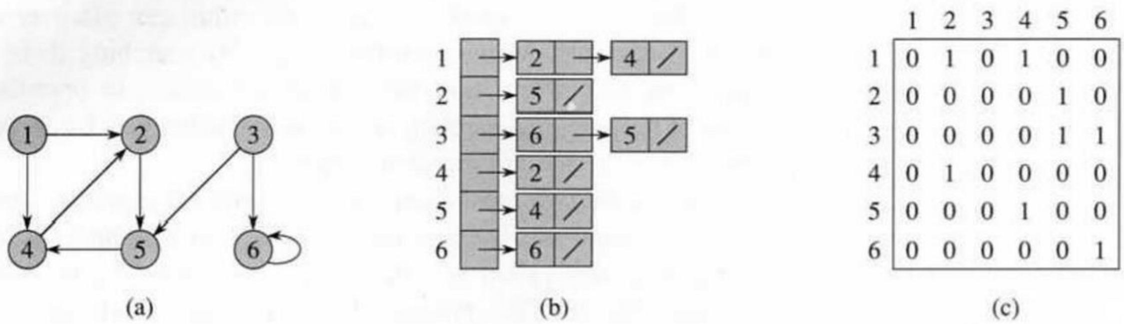
$$A[i][j] = \begin{cases} 1 & , \quad \text{if there is an edge between } i \text{ and } j \\ 0 & , \quad \text{otherwise} \end{cases}$$

2. **Adjacency List**



	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

Figure 22.1 Two representations of an undirected graph. **(a)** An undirected graph G having five vertices and seven edges. **(b)** An adjacency-list representation of G. **(c)** The adjacency-matrix representation of G.

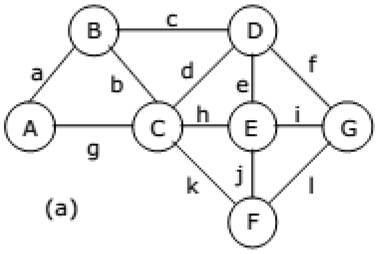


	1	2	3	4	5	6
1	0	1	0	1	0	0
2	0	0	0	0	1	0
3	0	0	0	0	1	1
4	0	1	0	0	0	0
5	0	0	0	1	0	0
6	0	0	0	0	0	1

Figure 22.2 Two representations of a directed graph. **(a)** A directed graph G having six vertices and eight edges. **(b)** An adjacency-list representation of G. **(c)** The adjacency-matrix representation of G.

3. **Incidence Matrix :** size of matrix is v*e , where v is total vertices , e is total edges

$$I[i][j] = \begin{cases} 1, & \text{if vertex } i \text{ is incident on edge } j \\ 0, & \text{otherwise} \end{cases}$$

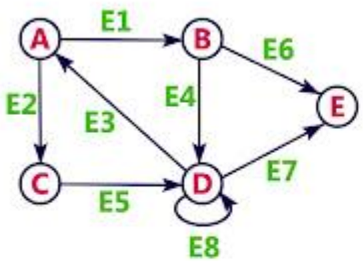


(b)

	a	b	c	d	e	f	g	h	i	j	k	l
A	1	0	0	0	0	0	1	0	0	0	0	0
B	1	1	1	0	0	0	0	0	0	0	0	0
C	0	1	0	1	0	0	1	1	0	0	1	0
D	0	0	1	1	1	1	0	0	0	0	0	0
E	0	0	0	0	1	0	0	1	1	1	0	0
F	0	0	0	0	0	0	0	0	0	1	1	1
G	0	0	0	0	0	1	0	0	1	0	0	1

Figure 6.5.4 Graph and its incidence matrix

Incidence Matrix for Undirected Graph



	E1	E2	E3	E4	E5	E6	E7	E8
A	1	1	-1	0	0	0	0	0
B	-1	0	0	1	0	1	0	0
C	0	-1	0	0	1	0	0	0
D	0	0	1	-1	-1	0	1	1
E	0	0	0	0	0	-1	-1	0

Incidence Matrix for Directed Graph

Introduction to Combinatorics

Combinatorics is a branch of mathematics that deals with counting, arranging, and analyzing discrete structures. It plays a crucial role in various fields including computer science, optimization, and statistical physics. Combinatorics provides tools and methods to solve problems related to the enumeration (counting of objects), existence (whether certain configurations are possible), construction (how to construct such configurations), and optimization (finding the best configuration).

Counting Techniques

1. **Basic Counting Principle**
   - **Description**: If there are n ways to do one thing and m ways to do another, then there are $n \times m$ ways to do both.
   - **Example**: If you have 3 shirts and 4 pairs of pants, there are $3 \times 4 = 12$ different outfits.
2. **Permutations**
   - **Description**: Permutations are arrangements of objects where the order matters.
   - **Formula**: The number of permutations of n distinct objects taken r at a time is $P(n, r) = \frac{n!}{(n-r)!}$.
   - **Example**: The number of ways to arrange 3 out of 5 books on a shelf is $P(5, 3) = \frac{5!}{(5-3)!} = 60$.
3. **Combinations**
   - **Description**: Combinations are selections of objects where the order does not matter.
   - **Formula**: The number of combinations of n distinct objects taken r at a time is $C(n, r) = \frac{n!}{r!(n-r)!}$.
   - **Example**: The number of ways to choose 3 out of 5 books is $C(5, 3) = \frac{5!}{3!(5-3)!} = 10$.
4. **Binomial Theorem**
   - **Description**: The binomial theorem provides a way to expand expressions of the form $(a + b)^n$.
   - **Formula**: $(a + b)^n = \sum_{k=0}^{n} C(n, k) a^{n-k} b^k$.
   - **Example**: Expanding $(x + y)^3$ gives $x^3 + 3x^2y + 3xy^2 + y^3$.
5. **Pigeonhole Principle**
   - **Description**: If n items are put into m containers with $n > m$, then at least one container must contain more than one item.
   - **Example**: If you have 10 pigeons and 9 pigeonholes, at least one pigeonhole must contain more than one pigeon.

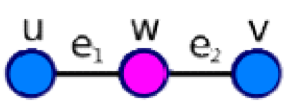
Q:- How many different mobile numbers can be formed of 10 digits that start with digit 9?

Graph Homeomorphism is a concept in graph theory that describes a relationship between graphs that can be transformed into each other through a series of edge subdivisions and contractions.

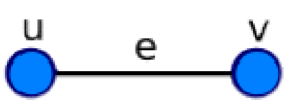
Key Concepts

1. **Subdivision of an Edge**:
   - Replacing an edge (u,v) with a new vertex w and edges (u,w) and (w,v).
   - This operation divides an edge into two edges by introducing a new vertex.
2. **Edge Contraction**:
   - The reverse process of subdivision, where two vertices u and v connected by an edge are merged into a single vertex, removing the edge (u,v).
   - This operation reduces the number of edges by merging two vertices connected by an edge.
3. **Homeomorphic Graphs**:
   - Two graphs G and H are homeomorphic if one can be transformed into the other through a sequence of edge subdivisions and contractions.
   - Homeomorphic graphs are considered equivalent in terms of their topological structure.

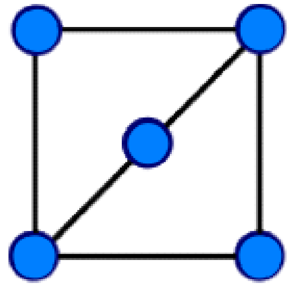
For example, the simple connected graph with two edges, e_1 {*u*,*w*} and e_2 {*w*,*v*}:



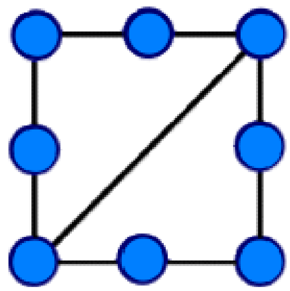
has a vertex (namely *w*) that can be smoothed away, resulting in:



In the following example, graph *G* and graph *H* are homeomorphic.

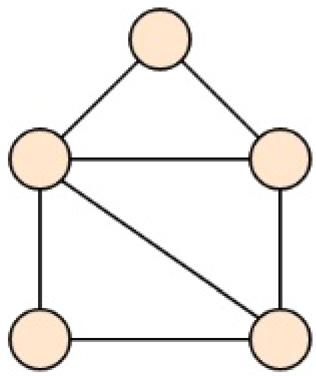


Graph *G*

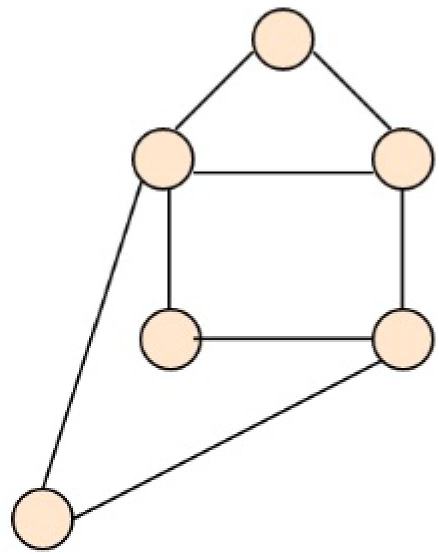


Graph *H*

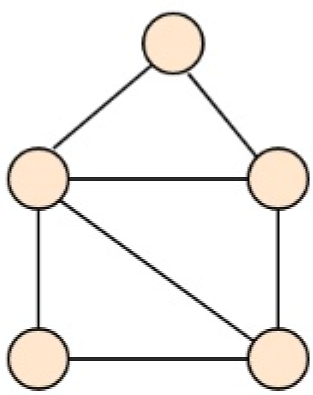
In The Following Examples G and G' are Homeomorphic Graphs & H and H' are Homeomorphic Graphs.



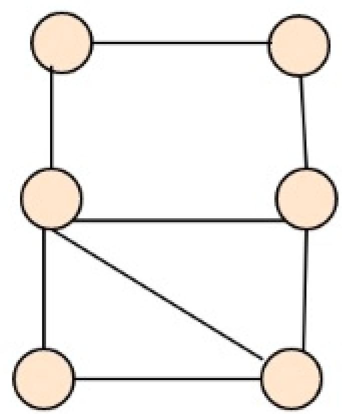
G



G'



H



H'